

ADASOLVER: EFFICIENT NEURAL OPERATOR INFERENCE VIA TEST-TIME ADAPTIVE SAMPLING

Anonymous authors

Paper under double-blind review

ABSTRACT

Neural operators provide efficient surrogates for partial differential equations (PDEs), but their inference cost is typically tied to a fixed spatial discretization or uniform subsampling. Such discretizations can waste computation when fine resolution is needed only in localized regions. However, changing the sampling density shifts the empirical measure in latent aggregation and can alter the continuous operator represented by a frozen point-based model. We introduce **AdaSolver**, a measure-aware framework for adaptive spatial computation at test time. We characterize sampling-induced bias and its propagation through Physics-Attention, then derive a sampling-density-consistent Transolver using self-normalized importance weighting. The corrected aggregation remains consistent with the reference measure, enabling adaptive point allocation guided by geometry and physics priors, coarse predictions, or a cache-aware learned refinement policy. We further extend variable-resolution learning to distribution-amortized training over both point counts and sampling densities. Experiments on five standard PDE benchmarks and three industrial aerodynamic prediction tasks show improved accuracy at matched point budgets, with solution-informed policies providing the strongest gains on the standard benchmarks. Airfoil and wing optimization further demonstrate the value of adaptive sampling for accelerating surrogate-based design. These results establish spatial sampling as a principled axis of test-time scaling for neural operators.

1 INTRODUCTION

Neural operators learn maps between function spaces and can amortize expensive numerical simulation across many physical configurations (Li et al., 2021; 2023b; Wu et al., 2024). Recently, transformer-based neural operators have improved the scalability and accuracy of operator learning across diverse PDEs and million-point geometries. They make such large-scale computation possible by compressing irregular meshes into latent tokens, distributing the computation, or decoupling global state estimation from pointwise decoding (Alkin et al., 2024; Luo et al., 2025; Zhou et al., 2026). In engineering applications, however, evaluation efficiency is as important as the ability to process large inputs. Yet how neural operators can flexibly allocate computation at inference remains largely unexplored.

Neural-operator training data are typically generated by numerical solvers on meshes that are uniform or designed for numerical convergence. Learned models are consequently evaluated under the same sampling distribution, which may ignore the strongly localized structure of many PDE solutions. Shocks, boundary layers, contact interfaces, and stress concentrations often occupy only a small part of the domain and require finer local resolution, whereas smooth regions can be represented with fewer samples. This creates a dilemma: preserving the training-time distribution wastes points in smooth regions, while reallocating them toward critical regions changes the input sampling distribution. Can a neural operator redistribute a fixed spatial budget without changing the operator it represents?

Existing efforts address neighboring capabilities. First, resolution generalization studies how neural operators extrapolate across grid sizes, point counts, or compatible discretizations while preserving the underlying sampling distribution (Li et al., 2021; 2023b; Liu et al., 2023; Alkin et al., 2024; Zhou et al., 2026; Gao et al., 2025). These methods make resolution changes feasible, but do not by

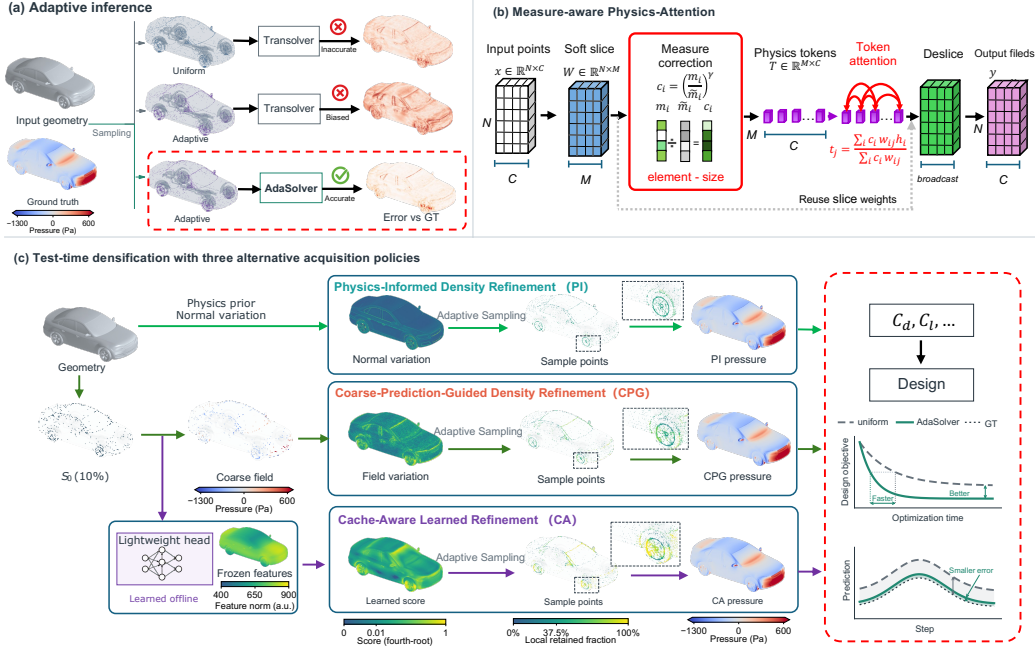


Figure 1: **AdaSolver overview.** (a) Adaptive inference with measure correction. (b) Corrected aggregation, token attention, and deslicing. (c) Geometry priors, coarse predictions, and a lightweight head on frozen features provide alternative refinement signals. The frozen operator evaluates the selected points and supplies aerodynamic quantities for design. The rightmost curves illustrate the design objectives schematically.

themselves provide a mechanism for actively changing the local sampling density. Second, adaptive sampling and refinement methods predict where a numerical mesh or point set should be refined, often as part of a solver pipeline or through a separately trained refinement model (Huang et al., 2021; Obiols-Sales et al., 2023; Rowbottom et al., 2024). They address spatial allocation, but do not specify how a frozen neural operator should preserve its target integral when that allocation changes the sampling measure. Third, test-time compute scaling increases inference effort through adaptive reasoning or iterative physical prediction (Snell et al., 2024; Lippe et al., 2023; Jenkins, 2026; Serrano et al., 2026). These works establish the value of allocating additional computation at inference, but leave open the spatial counterpart: reallocating input samples for operator evaluation while retaining the same learned map.

We introduce AdaSolver to address this gap. We view spatial resolution, controlled by the input sampling density, as a controllable inference resource. From a continuous-operator perspective, we characterize how the discrepancy between the inference-time sampling density and the reference distribution biases both the latent physical states and the predicted output function. Based on this analysis, we develop a sampling-density-consistent Transolver that uses self-normalized importance weighting to correct the sampling bias in physics- slice aggregation. This correction enables the model to construct an adaptive inference density from geometry or physics priors, a low-cost draft prediction, or a lightweight head attached to reusable intermediate features. We further incorporate these sampling strategies into training, supporting both geometry-amortized training across point sets and distribution-amortized training across sampling densities.

Figure 1 summarizes the measure correction, alternative refinement policies, and their use in surrogate-based design.

Our contributions are threefold:

- We formulate sampling density as the mechanism for controlling spatial test-time compute and derive how its deviation from the reference distribution biases the latent physical states and output function of a transformer neural operator.
- We propose a sampling-density-consistent Transolver that corrects physics-slice aggregation through self-normalized importance weighting, preserving the reference operator under inference-time density changes.
- We develop prior-guided, draft-guided, and cache-aware learned sampling strategies, and incorporate them into geometry-amortized and distribution-amortized training.

2 RELATED WORK

Neural operators on irregular geometries. Fourier neural operators learn resolution-transferable spectral mappings on regular grids, while Geo-FNO extends them to general geometries through learned coordinate deformations (Li et al., 2021; 2023a). GINO couples integral operators with a latent regular grid, and NUNO partitions non-uniform observations to reduce interpolation and computational costs (Li et al., 2023b; Liu et al., 2023). Transformer-based operators instead compress large point sets into latent representations before global interaction, as in GNOT, UPT, and Transolver (Hao et al., 2023; Alkin et al., 2024; Wu et al., 2024). Transolver++ and Transolver-3 further scale this design to million-scale and industrial-scale geometries (Luo et al., 2025; Zhou et al., 2026). These methods establish strong compatibility with irregular inputs and changing point counts. Our focus is complementary: we actively change the *local* sampling density at inference and ask whether the frozen model still represents the same continuous operator.

Discretization-consistent operator learning. Extending finite neural architectures to function spaces requires discrete reductions to converge to fixed continuous integrals as the discretization changes (Berner et al., 2026). Monte Carlo convolution corrects non-uniform point-cloud sampling with inverse-density weights, while QuadConv evaluates continuous kernels using quadrature on irregular PDE data (Hermosilla et al., 2018; Doherty et al., 2024). Related formulations construct discretization-invariant maps between neural fields and quadrature-aware normalization statistics (Wang & Golland, 2022; Kim et al., 2026). Nevertheless, resolution-independent parameters alone do not prevent errors under a mismatch between training and evaluation discretizations (Gao et al., 2025). We adopt the same function-space principle for latent point aggregation, but use it to support a deliberate inference-time change of sampling density rather than only passive resolution transfer.

Adaptive meshes and test-time refinement. Adaptive discretization concentrates resolution where numerical error is expected to be large. Learned approaches predict mesh density, refinement, or node relocation for numerical solvers (Huang et al., 2021; Obiols-Sales et al., 2023; Rowbottom et al., 2024), while MeshGraphNets integrates learned dynamics with mesh-based simulation (Pfaff et al., 2021). Test-time computation has also been increased through iterative denoising in PDE-Refiner, adaptive allocation or tokenization, and neural-operator composition (Lippe et al., 2023; Jenkins, 2026; Wang & Perdikaris, 2026; Serrano et al., 2026). These directions determine where or how additional inference computation should be used. AdaSolver addresses the complementary operator-level requirement: changing the spatial sampling density of a frozen point-based neural operator while keeping its reference continuous measure fixed.

3 METHOD

3.1 TRANSOLVER AS AN INTEGRAL OPERATOR

Transolver (Wu et al., 2024) approximates continuous integral operators through Physics-Attention. Let Ω denote the physical domain, and let ρ denote the sampling measure induced by the chosen grid or mesh discretization policy. A discretized example consists of pointwise samples $\{(x_i, a_i)\}_{i=1}^N \mapsto \{u_i\}_{i=1}^N$, where $x_i \sim \rho$, $a_i = a(x_i)$ is the input field, and $u_i = u(x_i)$ is the output field. A neural operator learns an operator $\mathcal{G} : a \mapsto u$ with a parameterized network $\mathcal{G}_\theta$.

Physics-Attention projects point features into M slice weights, applies attention to the resulting latent physical states, and broadcasts the updated states back to the points. Let $h_i \in \mathbb{R}^C$ denote the

C -dimensional feature embedding of (x_i, a_i) . The slice weights are

$$w_i = \text{Softmax}\left(\frac{\text{Linear}(h_i)}{\tau_0}\right) \in \mathbb{R}^M, \quad \sum_{j=1}^M w_{i,j} = 1, \quad (1)$$

where τ_0 is the temperature. The corresponding physics token is the normalized weighted aggregation

$$u_{s,j}^{(N)} = \frac{\frac{1}{N} \sum_{i=1}^N w_{i,j} h_i}{\frac{1}{N} \sum_{i=1}^N w_{i,j}} \xrightarrow{N \rightarrow \infty} u_{s,j}^\rho = \frac{\int_{\Omega} w_j(x) h(x) d\rho(x)}{\int_{\Omega} w_j(x) d\rho(x)}. \quad (2)$$

Writing $u_s^{(N)} = [u_{s,1}^{(N)}, \dots, u_{s,M}^{(N)}]^\top \in \mathbb{R}^{M \times C}$ and $q, k, v = \text{Linear}(u_s^{(N)})$, the discrete state attention and its continuous integral counterpart can be written together as

$$\begin{aligned} u'_{s,j} &= \sum_{t=1}^M \frac{\exp(q_j^\top k_t / \tau)}{\sum_{p=1}^M \exp(q_j^\top k_p / \tau)} v_t \\ &\xrightarrow{M \rightarrow \infty} \mathcal{A}[u_s^\rho](\xi'_s) \\ &:= \int_{\Omega_s} \frac{\exp(q(u_s^\rho(\xi'_s))^\top k(u_s^\rho(\xi_s)) / \tau)}{\int_{\Omega_s} \exp(q(u_s^\rho(\xi'_s))^\top k(u_s^\rho(\eta_s)) / \tau) d\eta_s} v(u_s^\rho(\xi_s)) d\xi_s. \end{aligned} \quad (3)$$

where $u_s^\rho(\xi_s)$ is the continuous physics-token field on the slice domain Ω_s , ξ'_s is a query location, and τ is the attention temperature. Finally, the updated tokens are mapped back to the point space through the slice weights,

$$h'_i = \sum_{j=1}^M w_{i,j} u'_{s,j}, \quad i = 1, \dots, N. \quad (4)$$

Thus, Physics-Attention replaces the $N \times N$ point interaction with operations of complexity $\mathcal{O}(NMC + M^2C)$ for $M \ll N$. During training, the point sets used by $\mathcal{G}_\theta$ are typically generated according to the prescribed measure ρ .

3.2 SAMPLING-DENSITY-CONSISTENT TRANSOLVER

A decoder-only transformer neural operator is typically trained and evaluated on point sets generated according to a prescribed sampling measure ρ , which is determined by the discretization used for data generation, such as a uniform grid or a geometry-dependent mesh-refinement policy. At inference time, we may instead sample from a different measure $\tilde{\rho}$ to obtain a different spatial description of the same physical domain. For example, aerodynamic geometries can be sampled more densely near surfaces or flow-sensitive structures and more sparsely in smooth exterior regions. However, because transformer neural operators approximate continuous operators through Monte Carlo integration, changing the sampling measure generally changes the limiting operator represented by the frozen model.

Let $\lambda = d\tilde{\rho}/d\rho > 0$ denote the density ratio. For a slice location $\xi_s \in \Omega_s$, define the slice-weighted probability measure

$$d\pi_{\xi_s}(x) := \frac{w(x, \xi_s)}{\int_{\Omega} w(y, \xi_s) d\rho(y)} d\rho(x). \quad (5)$$

Proposition 3.1 (Physics-token bias under a sampling-density shift). *For each $\xi_s \in \Omega_s$, the reference and shifted physics tokens satisfy*

$$u_{\xi_s}^{\tilde{\rho}} - u_{\xi_s}^{\rho} = \frac{\text{Cov}_{\pi_{\xi_s}}(\lambda, h)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (6)$$

Moreover, if $\mathcal{P}_{\xi_s}^\mu h := u_{\xi_s}^\mu(\xi_s)$ denotes the tokenization map under a sampling measure μ , then

$$\left\| \mathcal{P}_{\xi_s}^{\tilde{\rho}} - \mathcal{P}_{\xi_s}^{\rho} \right\|_{\text{op}} = \frac{\sqrt{\text{Var}_{\pi_{\xi_s}}(\lambda)}}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (7)$$

Let $\mathcal{T}$ denote the continuous Physics-Attention operator and define $\mathcal{G}_\mu(h) := \mathcal{T}[u_s^\mu]$. Write $\delta_s := u_s^{\tilde{\rho}} - u_s^\rho$ and let $R_{\delta_s} := \alpha_{u_s^{\tilde{\rho}}} / \alpha_{u_s^\rho}$ be the ratio between the shifted and reference attention kernels.

Theorem 3.2 (Propagation through continuous Physics-Attention). *The sampling-density bias propagates to the Physics-Attention output as*

$$\begin{aligned} \mathcal{G}_{\tilde{\rho}}(h)(x) - \mathcal{G}_\rho(h)(x) = & \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \alpha_{u_s^\rho}(\xi'_s, \xi_s) \left[(R_{\delta_s}(\xi'_s, \xi_s) - 1) W_v u_s^\rho(\xi_s) \right. \\ & \left. + R_{\delta_s}(\xi'_s, \xi_s) W_v \delta_s(\xi_s) \right] J_g(\xi_s) d\xi_s d\xi'_s. \end{aligned} \quad (8)$$

Here $\alpha_{u_s^\rho}$ is the reference attention kernel, W_v is the value projection, and J_g is the physical-to-slice Jacobian. The two terms respectively describe the redistribution of attention weights and the propagation of shifted value tokens.

The complete conditions and proofs of Proposition 3.1 and Theorem 3.2 are given in Appendix A.

These results identify the point-to-slice aggregation as the source of the sampling-density bias. The slice assignment $w_{i,j}$ is computed independently at each point, attention is subsequently applied among a fixed number of physics tokens, and deslicing maps the updated tokens back to each point. Only the physics-token construction aggregates over the sampled points. Consequently, for $x_i \sim \tilde{\rho}$, the original aggregation converges to

$$\frac{\sum_{i=1}^N w_{i,j} h_i}{\sum_{i=1}^N w_{i,j}} \xrightarrow{N \rightarrow \infty} \frac{\int_{\Omega} w_j(x) h(x) d\tilde{\rho}(x)}{\int_{\Omega} w_j(x) d\tilde{\rho}(x)} = u_{s,j}^{\tilde{\rho}}, \quad (9)$$

rather than the reference token $u_{s,j}^\rho$ learned under ρ .

We therefore correct the measure only when forming the physics tokens. For each inference point, let

$$c_i := \frac{d\rho}{d\tilde{\rho}}(x_i) = \lambda(x_i)^{-1}, \quad (10)$$

be its importance weight, or the corresponding inverse relative inclusion density for discrete sampling. The corrected aggregation is

$$\hat{u}_{s,j}^{(N)} := \frac{\sum_{i=1}^N c_i w_{i,j} h_i}{\sum_{i=1}^N c_i w_{i,j}} \xrightarrow{N \rightarrow \infty} \frac{\int_{\Omega} w_j(x) h(x) d\rho(x)}{\int_{\Omega} w_j(x) d\rho(x)} = u_{s,j}^\rho. \quad (11)$$

Because the estimator is self-normalized, only the relative density ratios are required. For finite samples, the implementation may replace c_i with c_i^β , where $\beta = 0$ recovers the original Transolver and $\beta = 1$ gives the measure-consistent correction above.

After the corrected physics tokens are obtained, we apply the original attention among physics tokens and the original deslice operation without further modification. For a Transolver with L layers, the same correction is applied whenever layer ℓ forms new physics tokens from its current point features $h_i^{(\ell)}$:

$$\hat{u}_{s,j}^{(\ell)} = \frac{\sum_{i=1}^N c_i w_{i,j}^{(\ell)} h_i^{(\ell)}}{\sum_{i=1}^N c_i w_{i,j}^{(\ell)}}, \quad \ell = 1, \dots, L. \quad (12)$$

Each layer then continues with its unchanged Physics-Attention and deslice operations. Thus, the correction follows the point-to-slice aggregation throughout the frozen network while preserving all learned projections and attention parameters.

3.3 ADAPTIVE TEST-TIME SAMPLING FOR TRANSOLVER

We next construct the inference-time sampling measure from geometry, a draft prediction, or intermediate backbone features.

Let $r_\psi : \Omega \rightarrow [0, \infty)$ be a nonnegative refinement score, where ψ denotes the information used to construct it, and define

$$\nu_\psi(x) := \frac{r_\psi(x)}{\int_{\Omega} r_\psi(y) d\rho(y)}, \quad \tilde{\rho}_{\gamma,\psi} := (1 - \gamma)\rho + \gamma(\nu_\psi * \rho), \quad \gamma \in [0, 1]. \quad (13)$$

Here $\nu_\psi * \rho$ denotes the reweighted empirical measure induced by the refinement score. The correction used in the point-to-slice aggregation is

$$c_{\gamma,\psi}(x) := \frac{d\rho}{d\tilde{\rho}_{\gamma,\psi}}(x) = \frac{1}{(1-\gamma) + \gamma\nu_\psi(x)}. \quad (14)$$

3.3.1 PHYSICS-INFORMED DENSITY REFINEMENT

The first policy constructs r_ψ from prior information about the geometry or the governing physical problem. Let $s_{\text{prior}}(x) \geq 0$ be a problem-dependent indicator whose large values identify regions that are expected to require finer spatial resolution. This indicator can be obtained from a known geometric feature, a numerical refinement rule, or a physical quantity available before inference. We set

$$\nu_{\text{prior}}(x) := \frac{s_{\text{prior}}(x)}{\int_{\Omega} s_{\text{prior}}(y) d\rho(y)} \quad (15)$$

and use it in equation 13 to sample the final point set.

This policy does not require a preliminary network evaluation; once the prior indicator is available, points are sampled from $\tilde{\rho}_{\gamma,\text{prior}}$ and inferred with the correction in equation 14.

3.3.2 COARSE-PREDICTION-GUIDED DENSITY REFINEMENT

This strategy follows the test-time refinement perspective of PDE-Refiner (Lippe et al., 2023) and the initial-prediction–difficulty-estimation pipeline of ATCA (Jenkins, 2026). When no reliable prior is available, we draw a small coarse set S_0 from ρ and evaluate the frozen operator on this set:

$$\hat{u}^{(0)} := \mathcal{G}_\theta(a|_{S_0}), \quad |S_0| \ll N, \quad (16)$$

where $a|_{S_0}$ denotes the input field restricted to the coarse points. From $\hat{u}^{(0)}$, we compute a non-negative local-variation score, for example by combining discrete estimates of the gradient and Laplacian:

$$s_{\text{coarse}}(x) := \Phi\left(\left\|\nabla\hat{u}^{(0)}(x)\right\|_2, \left\|\Delta\hat{u}^{(0)}(x)\right\|_2\right), \quad \Phi \geq 0, \quad (17)$$

where Φ is a fixed nonnegative combination and the derivatives are evaluated with the available mesh or point-cloud neighborhood. The score is transferred to candidate points, normalized as in equation 13, and used to sample $\tilde{\rho}_{\gamma,\text{coarse}}$.

The method works because the draft variation score directs the final spatial resolution toward regions where the field is changing rapidly.

3.3.3 CACHE-AWARE LEARNED REFINEMENT

The coarse prediction provides features correlated with local physical or geometric variation, but the subsequent computation of the first pass is otherwise discarded. We reuse it as follows. Let $S_1 = S_0 \cup B$ be the final point set, where B contains the newly allocated points. For the first Physics-Attention layer, let h_i^{in} be its pointwise input feature and $w_{i,j}^{(1)}$ its slice weight. Define

$$A_j(S) := \sum_{i \in S} c_i w_{i,j}^{(1)} h_i^{\text{in}}, \quad D_j(S) := \sum_{i \in S} c_i w_{i,j}^{(1)}. \quad (18)$$

Because the summands are pointwise, their values on S_0 are unchanged after adding B , and

$$A_j(S_1) = A_j(S_0) + A_j(B), \quad D_j(S_1) = D_j(S_0) + D_j(B). \quad (19)$$

so the coarse values can be cached and only the contributions from B need to be computed. We train a lightweight head H_ϕ with the frozen backbone, attach it to the first-block feature $h_i^{(1)}$, and define

$$\hat{u}_i^{\text{head}} = H_\phi(h_i^{(1)}), \quad s_{\text{head}}(x_i) := \Phi\left(\left\|\nabla\hat{u}^{\text{head}}(x_i)\right\|_2, \left\|\Delta\hat{u}^{\text{head}}(x_i)\right\|_2\right), \quad (20)$$

and use this score in equation 13. Token attention, deslicing, and later blocks still run on S_1 , because B changes the global token states.

3.4 DISTRIBUTION-AMORTIZED TRAINING

Using the measure-consistency property above, we extend the geometry-amortized training strategy of Transolver-3 (Zhou et al., 2026), which trains on random subsets of a high-resolution mesh, to distribution-amortized training. At step t , we draw a point count N_t from a prescribed range and choose a refinement configuration (γ_t, ψ_t) , giving

$$\rho_t := \tilde{\rho}_{\gamma_t, \psi_t}, \quad x_i^{(t)} \sim \rho_t, \quad i = 1, \dots, N_t, \quad S_t := \{x_i^{(t)}\}_{i=1}^{N_t}. \quad (21)$$

The configuration (γ_t, ψ_t) may be random or generated from a geometry- or physics-based refinement rule. We use the point-to-slice correction from equation 14 during the forward pass and optimize

$$\min_{\theta} \mathbb{E}_t \left[\ell \left(\mathcal{G}_{\theta} \left(a|_{S_t} \right), u|_{S_t} \right) \right]. \quad (22)$$

Because each corrected point-to-slice aggregation remains an estimator of the same ρ -based operator, changing N_t and ρ_t across training steps provides distributional augmentation without changing the operator target. Uniform variable-cardinality training is recovered when $\gamma_t = 0$.

4 EXPERIMENTS

We evaluate AdaSolver on five standard PDE benchmarks and three industrial design prediction tasks, covering both surface and volumetric fields on three-dimensional geometries. We further assess its efficiency in two CFD-validated geometry optimization tasks involving two-dimensional airfoils and three-dimensional wings.

4.1 BENCHMARKS

Our evaluation spans canonical PDE problems and representative industrial applications. The standard PDE suite comprises Elasticity, Plasticity, Airfoil, Pipe (Li et al., 2023a) and Darcy flow (Li et al., 2021). These public datasets are widely used to benchmark neural operators (Hao et al., 2023; Wu et al., 2024).

Governing equations, data splits, and input–output mappings are detailed in Appendix B.

For industrial applications, we consider ShapeNet-Car (Umetani & Bickel, 2018) and DriveAerML (Ashton et al., 2024) for automotive aerodynamics, and SuperWing (Yang et al., 2025) for three-dimensional wing design. In addition to physical-field prediction errors, we evaluate the accuracy of design-relevant aerodynamic quantities, including the drag coefficient C_D , lift coefficient C_L , and drag-to-lift ratio C_D/C_L . We also use the learned PDE surrogates to guide shape optimization of a two-dimensional airfoil and a three-dimensional wing, assessing computational efficiency and aerodynamic prediction accuracy during optimization.

4.2 IMPLEMENTATION AND EVALUATION

As described in equation 2 and equation 3, Transolver approximates a continuous integral operator through point aggregation and attention over latent physical states. With the number of physical states fixed, varying the spatial sampling budget changes the number of point samples used to estimate the aggregation integrals. For Airfoil, Pipe, Elasticity, and Plasticity, we adopt geometry-amortized training from Transolver-3 (Zhou et al., 2026). Given a full mesh of N points, we draw a point count N_t uniformly from $\{\text{round}(0.4N), \dots, N\}$ for each minibatch, then sample N_t mesh points uniformly without replacement. Darcy uses the existing backbone trained on full meshes. The trained backbone is frozen when evaluating different sampling policies and inference budgets.

For a consistent comparison, we treat all input discretizations as unstructured meshes and evaluate sampling budgets of 37.5%, 50%, and 75% of the full-mesh point count. Uniform subsampling of mesh points serves as the common baseline. We report relative L_2 errors for physical-field predictions and summarize performance across budgets using the relative area under the error–budget curve (rAUEC). Let b_k denote the retained fraction of mesh points, with $(b_1, b_2, b_3) = (0.375, 0.50, 0.75)$,

and let $E_{\text{method}}(b)$ and $E_{\text{uniform}}(b)$ denote mean test errors for a given sampling seed. Using trapezoidal integration, we compute

$$\text{rAUEC} := \frac{\sum_{k=1}^2 (b_{k+1} - b_k) [E_{\text{method}}(b_k) + E_{\text{method}}(b_{k+1})]}{\sum_{k=1}^2 (b_{k+1} - b_k) [E_{\text{uniform}}(b_k) + E_{\text{uniform}}(b_{k+1})]}. \quad (23)$$

Lower rAUEC is better, with uniform sampling normalized to 1. We report mean rAUEC and its sample standard deviation over three independent sampling runs with matched Uniform baselines. We also report end-to-end runtime for geometry optimization.

Airfoil uses the same 112 cases with reference $C_L > 0.01$ for field and C_D/C_L evaluation, with metric-specific sampling policies and absolute error for C_D/C_L . Elasticity uses 200 independent holdout cases.

For industrial benchmarks, ShapeNet-Car combines full-cloud velocity relative L_2 with half the surface-pressure relative L_2 error. DrivAerML uses surface-pressure relative L_2 , and SuperWing uses joint relative L_2 on the dataset-scaled pressure and skin-friction channels. The aerodynamic metrics are relative errors in C_D for the car datasets and C_D/C_L for SuperWing, with sampling policies selected separately for each metric. ShapeNet-Car and DrivAerML use 100 and 50 test geometries, respectively. SuperWing uses 2,871 cases for field evaluation and a fixed 2,563-case subset for C_D/C_L , selected by a calibration-derived lift-magnitude cutoff of approximately 0.071.

4.3 MAIN RESULTS

Standard Benchmarks. Table 1 compares physics-informed density refinement (PI), coarse-prediction-guided density refinement (CPG), and cache-aware learned refinement (CA) with uniform subsampling across five standard PDE benchmarks. Overall, adaptive sampling reduces rAUEC, showing that point density is a design variable for improving neural operator predictions under a fixed spatial budget. CPG generally improves upon PI, indicating that features of the predicted solution can guide point allocation more effectively than geometry and prescribed heuristics alone. CA achieves performance close to CPG on several benchmarks, supporting the learnability of the field-dependent signals used for density refinement. The differences are particularly clear on Darcy: PI provides little improvement over uniform sampling, whereas CPG and CA achieve similar, substantially lower rAUEC. Thus, even on a regular Cartesian grid, solution-informed density allocation can improve prediction accuracy. These results suggest that the benefit of adaptive sampling depends on how well its allocation signal reflects the solution field, beyond the geometric complexity of the domain.

Table 1: rAUEC ($\downarrow$) on standard PDE benchmarks over three seeds. Means appear above standard deviations. Best and second-best means are **bold** and underlined, respectively.

Method	Airfoil2D		Pipe	Darcy	Elasticity	Plasticity
	Field	C_D/C_L	Field	Field	Field	Field
PI	0.6177 ± 0.0187	0.8010 ± 0.2108	0.8711 ± 0.0061	0.9965 ± 0.0177	0.7280 ± 0.0083	0.8888 ± 0.0041
CPG	<u>0.6739</u> ± 0.0232	<u>0.4907</u> ± 0.3151	0.6719 ± 0.0098	<u>0.6419</u> ± 0.0049	0.4681 ± 0.0041	<u>0.8890</u> ± 0.0038
CA	0.6888 ± 0.0119	0.4128 ± 0.2384	<u>0.7051</u> ± 0.0116	0.6400 ± 0.0062	<u>0.5469</u> ± 0.0021	0.8897 ± 0.0049

Industrial Applications. Wind-tunnel testing informs vehicle and aircraft design by assessing aerodynamic performance through quantities such as drag and lift coefficients. Table 2 reports both full-field prediction errors and errors in aerodynamic quantities on ShapeNet-Car, DrivAerML, and SuperWing. The results show that all three methods substantially improve neural operator accuracy across sampling budgets. For aerodynamic quantities, concentrating samples in influential regions can match or even surpass full-mesh accuracy with only 37.5% of the points. This agrees with the physical intuition that some flow regions contribute more strongly to aerodynamic forces than others. Accurate predictions with fewer points can accelerate geometry optimization.

Table 2: rAUEC ($\downarrow$) on industrial benchmarks over three seeds. Means appear above standard deviations. Best and second-best means are **bold** and underlined, respectively.

Method	ShapeNet-Car		DrivAerML		SuperWing	
	Field	C_D	Field	C_D	Field	C_D/C_L
PI	0.58952 ± 0.00180	0.65606 ± 0.00168	0.88342 ± 0.00011	0.92455 ± 0.00149	0.75640 ± 0.00079	0.82456 ± 0.00539
CPG	0.55770 ± 0.00219	0.64743 ± 0.00454	0.82458 ± 0.00005	0.92093 ± 0.00148	0.67059 ± 0.00212	0.80259 ± 0.00761
CA	<u>0.55774</u> ± 0.00222	<u>0.64786</u> ± 0.00404	<u>0.82635</u> ± 0.00019	<u>0.92438</u> ± 0.00153	<u>0.71927</u> ± 0.00211	<u>0.81675</u> ± 0.00700

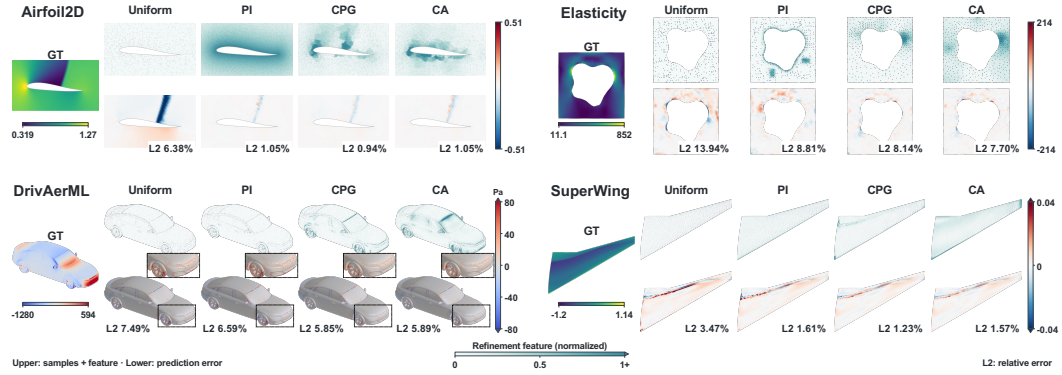


Figure 2: **Adaptive sampling and field prediction.** Four test cases at a 37.5% point budget. Upper rows show samples overlaid on refinement features; lower rows show signed prediction errors relative to ground truth. Each case uses shared color scales across methods. Annotations report relative L_2 field errors.

Figure 2 visualizes sampling locations, refinement features, and prediction errors for four test cases. Adaptive policies allocate points near localized field structure and reduce field error relative to uniform sampling in these examples.

4.4 INVERSE DESIGN

We use the frozen surrogate to optimize airfoil and wing geometries. The airfoil objective minimizes $(C_D/C_L)^2$, while the wing objective minimizes drag subject to predicted-lift and geometry constraints. We assess both tasks using independent CFD C_D/C_L . Timing covers online surrogate-based optimization; CFD validation is performed offline.

2D Airfoil. Figure 3 uses the Geo-FNO airfoil design case (Li et al., 2023a), with the same frozen operator, SLSQP optimizer, and geometry constraints for both methods. AdaSolver retains 18.75% of the points using CA sampling with seed 29 and fixes the indices after initialization. Five paired timing runs yield a $2.18\times$ speedup. The final CFD C_D/C_L values are 0.05879 for the full mesh and 0.05897 for AdaSolver, while relative C_D/C_L prediction error decreases from 3.26% to 0.46%.

3D Wing. Figure 4 compares 150 generations of CMA-ES on SuperWing case 2319 with seed 1, starting from the same geometry and flow condition. AdaSolver uses 37.5% of the full-mesh points, with sampling refreshed every 20 generations. The measured median completion times are 26.45 s for the full mesh and 14.18 s for AdaSolver, a $1.86\times$ speedup. Final CFD C_D/C_L decreases from 0.04283 to 0.04275, and the corresponding relative prediction error decreases from 3.82% to 3.30%. These results illustrate how reducing the cost of repeated surrogate evaluations can accelerate design while retaining comparable performance in CFD.

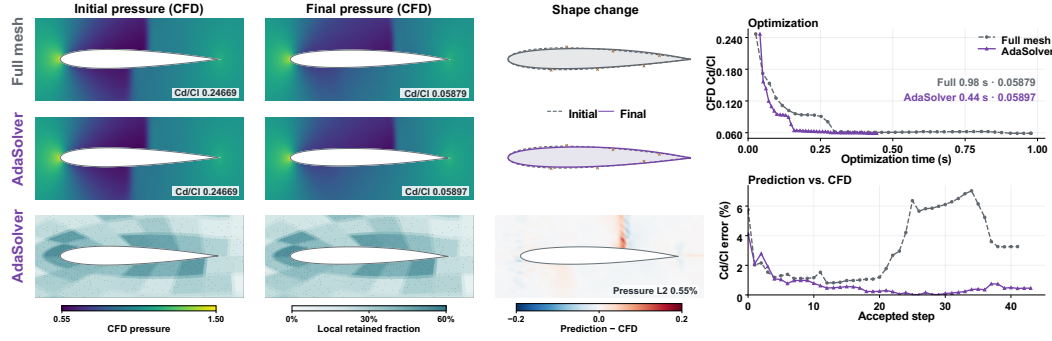


Figure 3: **Two-dimensional airfoil optimization.** Left: initial and final CFD pressure, geometry changes, and AdaSolver’s sampling distributions and pressure error. Right: CFD C_D/C_L against recorded optimization time, and relative C_D/C_L prediction error against accepted step. Both methods start from the same geometry.

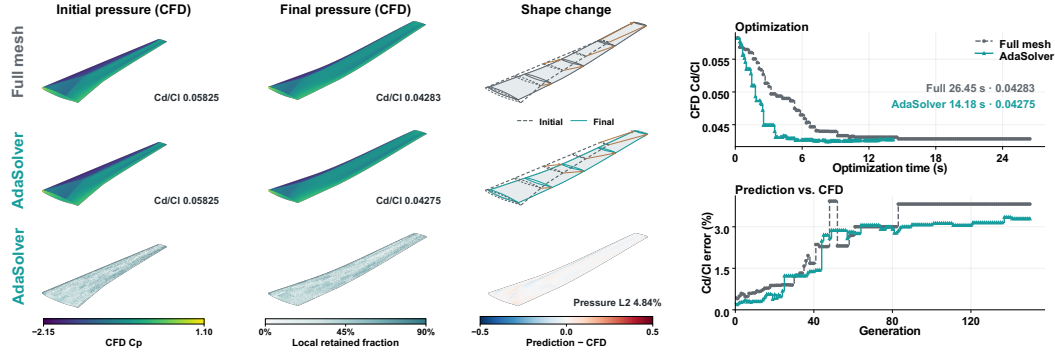


Figure 4: **SuperWing optimization.** Left: CFD pressure coefficients, geometry changes, and AdaSolver’s sampling distributions and pressure error. Right: CFD C_D/C_L against optimization time, and relative C_D/C_L prediction error over 150 CMA-ES generations. Independent CFD evaluates each retained geometry.

5 CONCLUSION

In this paper, we propose AdaSolver, a measure-aware framework for adaptive spatial computation in point-based neural operators. By correcting sampling-induced bias in latent aggregation, AdaSolver supports adaptive point allocation and distribution-amortized training while remaining consistent with the reference operator. Experiments on five standard PDE benchmarks, three industrial aerodynamic prediction tasks, and airfoil and wing optimization demonstrate improved accuracy at matched point budgets and faster surrogate-based design. This study establishes spatial sampling as an axis of test-time scaling for point-based neural operators, offering a direction for more efficient simulation and design in science and engineering.

REFERENCES

- Benedikt Alkin, Andreas Furst, Simon Schmid, Lukas Gruber, Markus Holzleitner, and Johannes Brandstetter. Universal physics transformers: A framework for efficiently scaling neural operators. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Neil Ashton, Charles Mockett, Marian Fuchs, Louis Fliessbach, Hendrik Hetmann, Thilo Knacke, Norbert Schonwald, Vangelis Skaperdas, Grigoris Fotiadis, Astrid Walle, Burkhard Hupertz, and Danielle Maddix. DrivAerML: High-fidelity computational fluid dynamics dataset for road-car external aerodynamics. *arXiv preprint arXiv:2408.11969*, 2024. doi: 10.48550/arXiv.2408.11969. URL <https://arxiv.org/abs/2408.11969>.

- Julius Berner, Miguel Liu-Schiaffini, Jean Kossaifi, Valentin Duruisseaux, Boris Bonev, Kamyar Azizzadenesheli, and Anima Anandkumar. Principled approaches for extending neural architectures to function spaces for operator learning. *Nature Machine Intelligence*, 8:1173–1181, 2026.
- Kevin Doherty, Colin Simpson, Stephen Becker, and Alireza Doostan. Quadconv: Quadrature-based convolutions with applications to non-uniform pde data compression. *Journal of Computational Physics*, 498:112636, 2024.
- Wenbo Gao, Rui Xu, Yuxin Deng, and Yang Liu. Discretization-invariance? on the discretization mismatch errors in neural operators. In *International Conference on Learning Representations*, 2025.
- Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. Gnot: A general neural operator transformer for operator learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 12556–12569. PMLR, 2023. URL <https://proceedings.mlr.press/v202/hao23c.html>.
- Pedro Hermosilla, Tobias Ritschel, Pere-Pau Vázquez, Àlvar Vinacua, and Timo Ropinski. Monte carlo convolution for learning on non-uniformly sampled point clouds. *ACM Transactions on Graphics*, 37(6), 2018.
- Kai Huang, Martin Krugener, Adam Brown, Felix Menhorn, Hans-Joachim Bungartz, and Dirk Hartmann. Machine learning-based optimal mesh generation in computational fluid dynamics. *arXiv preprint arXiv:2102.12923*, 2021.
- Ben Jenkins. Adaptive test-time compute allocation for neural pde solvers. In *International Conference on Learning Representations*, 2026.
- Byeong Joo Kim, Masaki Kawano, Yusuke Iwasawa, and Yutaka Matsuo. Quadnorm: Resolution-robust normalization for neural operators. *arXiv preprint arXiv:2605.07375*, 2026.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=c8P9NQVtmnO>.
- Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for pdes on general geometries. *Journal of Machine Learning Research*, 24(388):1–26, 2023a. URL <https://jmlr.org/papers/v24/23-0064.html>.
- Zongyi Li, Nikola Kovachki, Christopher Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Markus Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3d pdes. In *Advances in Neural Information Processing Systems*, volume 36, 2023b.
- Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Johannes Brandstetter. PDE-refiner: Achieving accurate long rollouts with neural PDE solvers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Sifan Liu, Zhongkai Hao, Chengyang Ying, Hang Su, Zhijie Cheng, and Jun Zhu. Nuno: A general framework for learning parametric pdes with non-uniform data. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Huakun Luo, Haixu Wu, Hang Zhou, Lei Xing, Yide Di, Jianmin Wang, and Mingsheng Long. Transolver++: An accurate neural solver for pdes on million-scale geometries. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- Oriol Obiols-Sales, Abhinav Vishnu, Nicholas Malaya, and Aparna Chandramowlishwaran. Adarnet: Deep learning predicts adaptive mesh refinement. In *International Conference on Parallel Processing*, pp. 524–534, 2023.

- Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.
- Jack Rowbottom, Georg Maierhofer, Thomas Deveney, Eike Mueller, Alberto Paganini, Katharina Schratz, Pietro Lio, Carola-Bibiane Schonlieb, and Chris Budd. G-adaptivity: Optimised graph-based mesh relocation for finite element methods. *arXiv preprint arXiv:2407.04516*, 2024.
- Louis Serrano, Jiachen Han, Edouard Oyallon, Samuel Ho, and Romain Morel. Test-time generalization for physics through neural operator splitting. *arXiv preprint arXiv:2602.00884*, 2026.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Nobuyuki Umetani and Bernd Bickel. Learning three-dimensional flow for interactive aerodynamic design. *ACM Transactions on Graphics*, 37(4), 2018. doi: 10.1145/3197517.3201325. URL <https://doi.org/10.1145/3197517.3201325>.
- Clinton J. Wang and Polina Golland. Discretization invariant networks for learning maps between neural fields. *arXiv preprint arXiv:2206.01178*, 2022.
- Hanwen Wang and Paris Perdikaris. Adaptive tokenization for vision transformer pde simulation. In *AI and PDE Workshop at the International Conference on Learning Representations*, 2026.
- Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024.
- Yunjia Yang, Weishao Tang, Mengxin Liu, Nils Thuerey, Yufei Zhang, and Haixin Chen. SuperWing: A comprehensive transonic wing dataset for data-driven aerodynamic design. *arXiv preprint arXiv:2512.14397*, 2025. doi: 10.48550/arXiv.2512.14397. URL <https://arxiv.org/abs/2512.14397>.
- Hang Zhou, Haixu Wu, Haonan Shangguan, Yuezhou Ma, Huikun Weng, Jianmin Wang, and Mingsheng Long. Transolver-3: Scaling up transformer solvers to industrial-scale geometries. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR, 2026. URL <https://arxiv.org/abs/2602.04940>.

A SAMPLING-MEASURE BIAS IN PHYSICS-ATTENTION

We analyze the effect of evaluating a frozen Physics-Attention layer under a sampling measure different from the one used during training. Let (Ω, ρ) be the physical domain equipped with the reference probability measure, let $h : \Omega \rightarrow \mathbb{R}^C$ be the point-feature field, and let Ω_s be a continuous slice domain of finite measure. A bounded measurable slice-weight function $w : \Omega \times \Omega_s \rightarrow [0, \infty)$ satisfies

$$\int_{\Omega_s} w(x, \xi_s) d\xi_s = 1, \quad \int_{\Omega} w(x, \xi_s) d\rho(x) > 0. \quad (\text{A.1})$$

For each $\xi_s \in \Omega_s$, define the slice-weighted probability measure

$$d\pi_{\xi_s}(x) := \frac{w(x, \xi_s)}{\int_{\Omega} w(y, \xi_s) d\rho(y)} d\rho(x). \quad (\text{A.2})$$

Let $\tilde{\rho}$ be the inference-time sampling measure. We assume $\tilde{\rho} \ll \rho$ and write

$$\lambda := \frac{d\tilde{\rho}}{d\rho} > 0 \quad \rho\text{-a.e.} \quad (\text{A.3})$$

The reference and shifted physics-token fields are

$$u_s^\rho(\xi_s) := \mathbb{E}_{\pi_{\xi_s}}[h], \quad u_s^{\tilde{\rho}}(\xi_s) := \frac{\mathbb{E}_{\pi_{\xi_s}}[\lambda h]}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (\text{A.4})$$

Since $\lambda > 0$ holds ρ -almost everywhere and w is bounded with positive ρ -mass,

$$\mathbb{E}_{\pi_{\xi_s}}[\lambda] = \frac{\int_{\Omega} w(x, \xi_s) d\tilde{\rho}(x)}{\int_{\Omega} w(x, \xi_s) d\rho(x)} \in (0, \infty).$$

Hence, $u_s^{\tilde{\rho}}$ is well defined, and its discrepancy from u_s^ρ reflects a change in the limiting tokenization operator rather than finite-sample quadrature error.

Proposition A.1 (Exact bias of a physics token). *Fix $\xi_s \in \Omega_s$. Assume $h \in L^2(\pi_{\xi_s}; \mathbb{R}^C)$ and $\lambda h \in L^1(\pi_{\xi_s}; \mathbb{R}^C)$. Then*

$$u_s^{\tilde{\rho}}(\xi_s) - u_s^\rho(\xi_s) = \frac{\text{Cov}_{\pi_{\xi_s}}(\lambda, h)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} = \mathbb{E}_{\pi_{\xi_s}} \left[\left(\frac{\lambda}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} - 1 \right) h \right], \quad (\text{A.5})$$

where

$$\text{Cov}_{\pi_{\xi_s}}(\lambda, h) := \mathbb{E}_{\pi_{\xi_s}}[(\lambda - \mathbb{E}_{\pi_{\xi_s}}[\lambda])(h - \mathbb{E}_{\pi_{\xi_s}}[h])] \in \mathbb{R}^C. \quad (\text{A.6})$$

Furthermore, if $\lambda \in L^2(\pi_{\xi_s})$, define the bounded linear operators from $L^2(\pi_{\xi_s}; \mathbb{R}^C)$ to $\mathbb{R}^C$ by

$$\mathcal{P}_{\xi_s}^\rho f := \mathbb{E}_{\pi_{\xi_s}}[f], \quad \mathcal{P}_{\xi_s}^{\tilde{\rho}} f := \frac{\mathbb{E}_{\pi_{\xi_s}}[\lambda f]}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (\text{A.7})$$

Their operator-norm discrepancy is exactly

$$\|\mathcal{P}_{\xi_s}^{\tilde{\rho}} - \mathcal{P}_{\xi_s}^\rho\|_{\text{op}} = \left\| \frac{\lambda}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} - 1 \right\|_{L^2(\pi_{\xi_s})} = \frac{\sqrt{\text{Var}_{\pi_{\xi_s}}(\lambda)}}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (\text{A.8})$$

Proof. By equation A.4,

$$u_s^{\tilde{\rho}}(\xi_s) - u_s^\rho(\xi_s) = \frac{\mathbb{E}_{\pi_{\xi_s}}[\lambda h] - \mathbb{E}_{\pi_{\xi_s}}[\lambda] \mathbb{E}_{\pi_{\xi_s}}[h]}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} = \frac{\text{Cov}_{\pi_{\xi_s}}(\lambda, h)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}, \quad (\text{A.9})$$

which also equals the second expression in equation A.5.

For the operator statement, set

$$r_{\xi_s} := \frac{\lambda}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} - 1. \quad (\text{A.10})$$

Then, for every $f \in L^2(\pi_{\xi_s}; \mathbb{R}^C)$,

$$(\mathcal{P}_{\xi_s}^{\tilde{\rho}} - \mathcal{P}_{\xi_s}^\rho) f = \mathbb{E}_{\pi_{\xi_s}}[r_{\xi_s} f]. \quad (\text{A.11})$$

Cauchy–Schwarz gives

$$\|\mathbb{E}_{\pi_{\xi_s}}[r_{\xi_s} f]\|_2 \leq \|r_{\xi_s}\|_{L^2(\pi_{\xi_s})} \|f\|_{L^2(\pi_{\xi_s}; \mathbb{R}^C)}, \quad (\text{A.12})$$

so the operator norm is at most $\|r_{\xi_s}\|_{L^2(\pi_{\xi_s})}$. If $r_{\xi_s} \neq 0$, choose a unit vector $e \in \mathbb{R}^C$ and

$$f^\star(x) := \frac{r_{\xi_s}(x)}{\|r_{\xi_s}\|_{L^2(\pi_{\xi_s})}} e. \quad (\text{A.13})$$

Then $\|f^\star\|_{L^2(\pi_{\xi_s}; \mathbb{R}^C)} = 1$ and

$$(\mathcal{P}_{\xi_s}^{\tilde{\rho}} - \mathcal{P}_{\xi_s}^\rho) f^\star = \|r_{\xi_s}\|_{L^2(\pi_{\xi_s})} e, \quad (\text{A.14})$$

which attains the upper bound. The case $r_{\xi_s} = 0$ is immediate. Finally,

$$\|r_{\xi_s}\|_{L^2(\pi_{\xi_s})}^2 = \frac{\mathbb{E}_{\pi_{\xi_s}}[(\lambda - \mathbb{E}_{\pi_{\xi_s}}[\lambda])^2]}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]^2} = \frac{\text{Var}_{\pi_{\xi_s}}(\lambda)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]^2}, \quad (\text{A.15})$$

which proves equation A.8. $\square$

A.1 PROPAGATION THROUGH CONTINUOUS PHYSICS-ATTENTION

We first recall the continuous formulation leading to equation 13 of Transolver (Wu et al., 2024). Let $g : \Omega \rightarrow \Omega_s$ be a diffeomorphism from the physical domain to the slice domain, with

$$J_g(\xi_s) := |\det(\nabla_{\xi_s} g^{-1}(\xi_s))|. \quad (\text{A.16})$$

For a bounded physics-token field $u_s : \Omega_s \rightarrow \mathbb{R}^C$, the slice-to-slice Physics-Attention kernel is

$$\begin{aligned} \ell_{u_s}(\xi'_s, \xi_s) &:= \frac{(W_q u_s(\xi'_s))^\top W_k u_s(\xi_s)}{\tau}, \\ \alpha_{u_s}(\xi'_s, \xi_s) &:= \frac{\exp(\ell_{u_s}(\xi'_s, \xi_s))}{\int_{\Omega_s} \exp(\ell_{u_s}(\xi'_s, \zeta_s)) d\zeta_s}, \\ \kappa_{ss, u_s}(\xi'_s, \xi_s) &:= \alpha_{u_s}(\xi'_s, \xi_s) W_v, \end{aligned} \quad (\text{A.17})$$

where $W_q, W_k \in \mathbb{R}^{d \times C}$, $W_v \in \mathbb{R}^{C \times C}$, and $\tau > 0$ are fixed. Thus, κ_{ss, u_s} is the matrix-valued kernel that performs attention among physics tokens. After expressing the physical-to-slice kernel as a weighted combination of κ_{ss, u_s} , the continuous Physics-Attention operator becomes

$$\begin{aligned} \mathcal{T}[u_s](x) &:= \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \kappa_{ss, u_s}(\xi'_s, \xi_s) u_s(\xi_s) J_g(\xi_s) d\xi_s d\xi'_s \\ &= \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \alpha_{u_s}(\xi'_s, \xi_s) W_v u_s(\xi_s) J_g(\xi_s) d\xi_s d\xi'_s. \end{aligned} \quad (\text{A.18})$$

This is the deslice-attention-token factorization in the continuous derivation of Transolver. If the slice domain is represented by M tokens $s_1, \dots, s_M$, with $q_j = W_q s_j$, $k_j = W_k s_j$, and $v_j = W_v s_j$, its Monte Carlo discretization is

$$h'_i = \sum_{j=1}^M w_{i,j} \sum_{t=1}^M \frac{\exp(q_j^\top k_t / \tau)}{\sum_{p=1}^M \exp(q_j^\top k_p / \tau)} v_t, \quad i = 1, \dots, N, \quad (\text{A.19})$$

which is equation 13 of Transolver under the simplifying choice $J_g \equiv 1$. For the token fields in equation A.4, we write $\mathcal{G}_\rho(h) := \mathcal{T}[u_\rho^p]$ and $\mathcal{G}_{\tilde{\rho}}(h) := \mathcal{T}[u_{\tilde{\rho}}^p]$. The map g , the slice weight w , and the projection matrices are held fixed when the sampling measure changes.

Lemma A.2 (Exact output perturbation for a token shift). *Let $u_s, \delta_s \in L^\infty(\Omega_s; \mathbb{R}^C)$. The logit perturbation induced by replacing u_s with $u_s + \delta_s$ is*

$$\begin{aligned} \Delta \ell_{\delta_s}(\xi'_s, \xi_s) &:= \ell_{u_s + \delta_s}(\xi'_s, \xi_s) - \ell_{u_s}(\xi'_s, \xi_s) \\ &= \frac{(W_q \delta_s(\xi'_s))^\top W_k u_s(\xi_s) + (W_q u_s(\xi'_s))^\top W_k \delta_s(\xi_s) + (W_q \delta_s(\xi'_s))^\top W_k \delta_s(\xi_s)}{\tau}. \end{aligned} \quad (\text{A.20})$$

For each query ξ'_s , define

$$R_{\delta_s}(\xi'_s, \xi_s) := \frac{\exp(\Delta \ell_{\delta_s}(\xi'_s, \xi_s))}{\int_{\Omega_s} \alpha_{u_s}(\xi'_s, \zeta_s) \exp(\Delta \ell_{\delta_s}(\xi'_s, \zeta_s)) d\zeta_s}. \quad (\text{A.21})$$

Then $\alpha_{u_s + \delta_s} = \alpha_{u_s} R_{\delta_s}$, and

$$\begin{aligned} \mathcal{T}[u_s + \delta_s](x) - \mathcal{T}[u_s](x) &= \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \alpha_{u_s}(\xi'_s, \xi_s) \left[(R_{\delta_s}(\xi'_s, \xi_s) - 1) W_v u_s(\xi_s) \right. \\ &\quad \left. + R_{\delta_s}(\xi'_s, \xi_s) W_v \delta_s(\xi_s) \right] J_g(\xi_s) d\xi_s d\xi'_s. \end{aligned} \quad (\text{A.22})$$

The two terms are the attention-redistribution bias and the value-token bias, respectively.

Proof. By equation A.20,

$$\begin{aligned} \alpha_{u_s + \delta_s}(\xi'_s, \xi_s) &= \frac{\exp(\ell_{u_s}(\xi'_s, \xi_s)) \exp(\Delta \ell_{\delta_s}(\xi'_s, \xi_s))}{\int_{\Omega_s} \exp(\ell_{u_s}(\xi'_s, \zeta_s)) \exp(\Delta \ell_{\delta_s}(\xi'_s, \zeta_s)) d\zeta_s} \\ &= \alpha_{u_s}(\xi'_s, \xi_s) R_{\delta_s}(\xi'_s, \xi_s). \end{aligned} \quad (\text{A.23})$$

Substituting this identity and $u_s + \delta_s$ into equation A.18, then subtracting $\mathcal{T}[u_s]$, gives equation A.22. $\square$

Theorem A.3 (Exact propagation of sampling-measure bias). *Assume the conditions of Proposition A.1 hold for every $\xi_s \in \Omega_s$, and define the token perturbation*

$$\delta_s^{\tilde{\rho}}(\xi_s) := u_s^{\tilde{\rho}}(\xi_s) - u_s^{\rho}(\xi_s) = \frac{\text{Cov}_{\pi_{\xi_s}}(\lambda, h)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]}. \quad (\text{A.24})$$

If $u_s^{\rho}, \delta_s^{\tilde{\rho}} \in L^{\infty}(\Omega_s; \mathbb{R}^C)$, then

$$\begin{aligned} \mathcal{G}_{\tilde{\rho}}(h)(x) - \mathcal{G}_{\rho}(h)(x) &= \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \alpha_{u_s^{\rho}}(\xi'_s, \xi_s) \left[(R_{\delta_s^{\tilde{\rho}}}(\xi'_s, \xi_s) - 1) W_v u_s^{\rho}(\xi_s) \right. \\ &\quad \left. + R_{\delta_s^{\tilde{\rho}}}(\xi'_s, \xi_s) W_v \frac{\text{Cov}_{\pi_{\xi_s}}(\lambda, h)}{\mathbb{E}_{\pi_{\xi_s}}[\lambda]} \right] J_g(\xi_s) d\xi_s d\xi'_s, \end{aligned} \quad (\text{A.25})$$

where $R_{\delta_s^{\tilde{\rho}}}$ is given by equation A.21 with base field $u_s = u_s^{\rho}$. Thus, the exact output bias is the sum of the change in attention weights and the direct propagation of the biased value tokens.

Proof. Equation A.24 follows from Proposition A.1. Applying Lemma A.2 with $u_s = u_s^{\rho}$ and $\delta_s = \delta_s^{\tilde{\rho}}$ gives equation A.25. $\square$

We next consider a small change of sampling measure,

$$d\rho_{\varepsilon}(x) = (1 + \varepsilon\eta(x)) d\rho(x), \quad \mathbb{E}_{\rho}[\eta] = 0, \quad (\text{A.26})$$

where $\eta \in L^{\infty}(\rho)$ and $|\varepsilon|$ is small enough that $1 + \varepsilon\eta > 0$. Let $u_s^{\rho_{\varepsilon}}$ be the token field under ρ_{ε} . Proposition A.1 gives

$$u_s^{\rho_{\varepsilon}}(\xi_s) - u_s^{\rho}(\xi_s) = \frac{\varepsilon \text{Cov}_{\pi_{\xi_s}}(\eta, h)}{1 + \varepsilon \mathbb{E}_{\pi_{\xi_s}}[\eta]} = \varepsilon \text{Cov}_{\pi_{\xi_s}}(\eta, h) + \mathcal{O}(\varepsilon^2), \quad (\text{A.27})$$

uniformly in ξ_s . The corresponding first-order change of the attention logit is

$$\begin{aligned} \ell_{\eta}^{(1)}(\xi'_s, \xi_s) &:= \frac{1}{\tau} \left[(W_q \text{Cov}_{\pi_{\xi'_s}}(\eta, h))^{\top} W_k u_s^{\rho}(\xi_s) \right. \\ &\quad \left. + (W_q u_s^{\rho}(\xi'_s))^{\top} W_k \text{Cov}_{\pi_{\xi_s}}(\eta, h) \right]. \end{aligned} \quad (\text{A.28})$$

Theorem A.4 (First-order sampling-measure bias). *Assume $h \in L^{\infty}(\rho; \mathbb{R}^C)$, $J_g \in L^{\infty}(\Omega_s)$, and the preceding conditions on w and η . Then, in $L^{\infty}(\Omega; \mathbb{R}^C)$,*

$$\mathcal{G}_{\rho_{\varepsilon}}(h) - \mathcal{G}_{\rho}(h) = \varepsilon \mathcal{S}_{\rho, h}[\eta] + \mathcal{O}(\varepsilon^2), \quad (\text{A.29})$$

where

$$\begin{aligned} \mathcal{S}_{\rho, h}[\eta](x) &= \int_{\Omega_s} w(x, \xi'_s) \int_{\Omega_s} \alpha_{u_s^{\rho}}(\xi'_s, \xi_s) \left\{ W_v \text{Cov}_{\pi_{\xi_s}}(\eta, h) \right. \\ &\quad \left. + \left[\ell_{\eta}^{(1)}(\xi'_s, \xi_s) - \int_{\Omega_s} \alpha_{u_s^{\rho}}(\xi'_s, \zeta_s) \ell_{\eta}^{(1)}(\xi'_s, \zeta_s) d\zeta_s \right] W_v u_s^{\rho}(\xi_s) \right\} J_g(\xi_s) d\xi_s d\xi'_s. \end{aligned} \quad (\text{A.30})$$

Consequently,

$$\lim_{\varepsilon \rightarrow 0} \frac{\|\mathcal{G}_{\rho_{\varepsilon}}(h) - \mathcal{G}_{\rho}(h)\|_{L^{\infty}(\Omega)}}{|\varepsilon|} = \|\mathcal{S}_{\rho, h}[\eta]\|_{L^{\infty}(\Omega)}. \quad (\text{A.31})$$

If $\mathcal{S}_{\rho, h}[\eta] \neq 0$, then the output bias is $\Theta(|\varepsilon|)$.

Proof. The boundedness assumptions make the remainder in equation A.27 uniform over Ω_s . By bilinearity of the query-key logit, equation A.20 therefore gives

$$\Delta \ell_{u_s^{\rho_{\varepsilon}} - u_s^{\rho}}(\xi'_s, \xi_s) = \varepsilon \ell_{\eta}^{(1)}(\xi'_s, \xi_s) + \mathcal{O}(\varepsilon^2) \quad (\text{A.32})$$

uniformly on $\Omega_s \times \Omega_s$. Expanding the numerator and denominator of equation A.21 yields

$$R_{u_s^{\rho_\varepsilon} - u_s^\rho}(\xi'_s, \xi_s) = 1 + \varepsilon \left[\ell_\eta^{(1)}(\xi'_s, \xi_s) - \int_{\Omega_s} \alpha_{u_s^\rho}(\xi'_s, \zeta_s) \ell_\eta^{(1)}(\xi'_s, \zeta_s) d\zeta_s \right] + \mathcal{O}(\varepsilon^2), \quad (\text{A.33})$$

also uniformly. Moreover, equation A.27 implies

$$W_v(u_s^{\rho_\varepsilon}(\xi_s) - u_s^\rho(\xi_s)) = \varepsilon W_v \text{Cov}_{\pi_{\xi_s}}(\eta, h) + \mathcal{O}(\varepsilon^2). \quad (\text{A.34})$$

Substituting equation A.33 and equation A.34 into Lemma A.2 and collecting the coefficient of ε gives equation A.30. Since J_g is bounded and $\int_{\Omega_s} w(x, \xi'_s) d\xi'_s = 1$, the integrated remainder is $\mathcal{O}(\varepsilon^2)$ in $L^\infty(\Omega)$, proving equation A.29.

Finally, the reverse triangle inequality gives

$$\left| \frac{\|\mathcal{G}_{\rho_\varepsilon}(h) - \mathcal{G}_\rho(h)\|_{L^\infty(\Omega)}}{|\varepsilon|} - \|\mathcal{S}_{\rho, h}[\eta]\|_{L^\infty(\Omega)} \right| = \mathcal{O}(|\varepsilon|), \quad (\text{A.35})$$

which proves equation A.31 and the stated $\Theta(|\varepsilon|)$ order. $\square$

The Jacobian J_g remains in every bias term. Setting $J_g \equiv 1$ recovers the simplifying assumption used by Transolver when deriving its discrete Physics-Attention update.

B IMPLEMENTATION DETAILS

We describe the governing equations, data splits, and forward operators for the benchmarks used in our experiments. All sampling budgets are measured relative to the reference point sets specified below.

B.1 STANDARD BENCHMARKS

Elasticity and Plasticity follow the solid-mechanics benchmarks in Geo-FNO (Li et al., 2023a). Both satisfy momentum balance,

$$\rho_s \frac{\partial^2 \mathbf{u}}{\partial t^2} = \nabla \cdot \boldsymbol{\sigma}, \quad (\text{B.1})$$

where ρ_s is material density, $\mathbf{u}$ is displacement, and $\boldsymbol{\sigma}$ is stress. The two tasks use different constitutive laws to describe the material response.

Elasticity. This task predicts the equilibrium stress in a unit square containing an irregular central hole. The bottom edge is fixed, and the top edge is pulled upward. The material follows the incompressible Rivlin–Saunders model, with strain energy

$$W = C_1(I_1 - 3) + C_2(I_2 - 3), \quad (\text{B.2})$$

where I_1 and I_2 are deformation invariants, $C_1 = 1.863 \times 10^5$, and $C_2 = 9.79 \times 10^3$. The hole radius varies between 0.2 and 0.4, and each geometry contains 972 unstructured points. The operator maps the geometry to the von Mises stress field, $\mathcal{G}_{\text{elas}} : \Omega \mapsto \sigma_{\text{vM}}(\cdot)$. We use 1,000 training, 800 validation, and 200 test geometries.

Plasticity. This task models a metal block compressed by a rigid, frictionless die with profile S_d . It follows equation B.1 with an elastoplastic stress law,

$$\boldsymbol{\sigma} = \mathbb{C} : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p), \quad (\text{B.3})$$

where $\mathbb{C}$ is the elastic stiffness tensor, and $\boldsymbol{\varepsilon}$ and $\boldsymbol{\varepsilon}^p$ are total and plastic strain. The material obeys the von Mises yield criterion, with Young’s modulus 200 GPa, Poisson’s ratio 0.3, yield strength 70 MPa, and density 7850 kg m^{-3} . The $50 \text{ mm} \times 15 \text{ mm}$ block is fixed at the bottom and compressed at 3 m s^{-1} . We use 800 training, 107 validation, and 80 test cases. Each case contains a 101×31 mesh and 20 time steps. The operator maps the die profile to the displacement sequence, $\mathcal{G}_{\text{plas}} : S_d \mapsto \{\mathbf{u}(\cdot, t_j)\}_{j=1}^{20}$, predicting both displacement components at all time steps in one pass.

Airfoil. The Airfoil benchmark (Li et al., 2023a) describes transonic flow around deformations of a NACA-0012 airfoil. The steady inviscid flow satisfies the compressible Euler equations,

$$\begin{aligned} \nabla \cdot (\rho_f \mathbf{v}) &= 0, \quad \nabla \cdot (\rho_f \mathbf{v} \otimes \mathbf{v} + p \mathbf{I}) = 0, \\ \nabla \cdot [(\mathcal{E} + p)\mathbf{v}] &= 0, \quad p = (\gamma - 1) \left(\mathcal{E} - \frac{1}{2} \rho_f \|\mathbf{v}\|^2 \right), \end{aligned} \quad (\text{B.4})$$

where ρ_f , $\mathbf{v}$, p , and $\mathcal{E}$ denote fluid density, velocity, pressure, and total energy per unit volume. The freestream conditions are $\rho_\infty = p_\infty = 1$, $M_\infty = 0.8$, and zero angle of attack, with $\gamma = 1.4$ and no penetration at the airfoil surface. Each case uses a 221×51 body-fitted mesh with 11,271 points. The dataset split contains 1,000 training and 200 test cases. Our operator maps mesh coordinates to pressure, $\mathcal{G}_{\text{airfoil}} : \Omega \mapsto p(\cdot)$, from which surface integration provides C_D/C_L .

Pipe. The Pipe benchmark (Li et al., 2023a) considers incompressible viscous flow through two-dimensional pipes with varying centerlines. The governing Navier–Stokes equations are

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\nabla p + \nu \Delta \mathbf{v}, \quad \nabla \cdot \mathbf{v} = 0, \quad (\text{B.5})$$

where p is pressure divided by the constant density and $\nu = 0.005$. The inlet has a parabolic velocity profile with unit peak speed; the walls satisfy no-slip conditions, and the outlet has a free boundary condition. Pipes have length 10 and width 1, with centerlines parameterized by piecewise cubic polynomials. The dataset contains 1,000 training and 200 test geometries, each discretized on a 129×129 mesh with 16,641 points. The forward operator $\mathcal{G}_{\text{pipe}} : \Omega \mapsto v_x(\cdot)$ maps the two-dimensional mesh coordinates to the steady horizontal velocity field.

Darcy. The Darcy benchmark (Li et al., 2021) models steady flow through a heterogeneous porous medium on $\Omega = (0, 1)^2$,

$$-\nabla \cdot (a(x) \nabla p(x)) = 1 \quad \text{in } \Omega, \quad p = 0 \quad \text{on } \partial\Omega, \quad (\text{B.6})$$

where a is the permeability coefficient and p is pressure. The coefficient takes values 3 and 12 according to the sign of a Gaussian random field. Solutions are generated on a 421×421 grid and subsampled to an 85×85 reference grid of 7,225 points. We use 1,000 training and 200 test cases. The operator $\mathcal{G}_{\text{Darcy}} : a(\cdot) \mapsto p(\cdot)$ maps the permeability field to pressure.

B.2 INDUSTRIAL APPLICATIONS

The industrial tasks predict aerodynamic fields on three-dimensional vehicle and wing geometries. Drag and lift are obtained by integrating the predicted surface forces.

ShapeNet-Car. ShapeNet-Car (Umetani & Bickel, 2018; Wu et al., 2024) contains 889 car geometries simulated at a driving speed of 72 km h^{-1} . The underlying three-dimensional flow is governed by the incompressible Navier–Stokes equations,

$$\nabla \cdot \mathbf{v} = 0, \quad \rho_f \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \Delta \mathbf{v}, \quad (\text{B.7})$$

where μ is dynamic viscosity. Each case contains 32,186 points covering the vehicle surface and surrounding volume. We use 710 training, 79 validation, and 100 test geometries. The operator maps coordinates, signed distance, and surface normals to velocity and pressure, $\mathcal{G}_{\text{car}} : (\mathbf{x}, \text{SDF}, \mathbf{n}) \mapsto (\mathbf{v}, p)$. Velocity is predicted over the full point cloud and pressure on the surface; both fields contribute to drag estimation.

DrivAerML. DrivAerML (Ashton et al., 2024) provides high-fidelity automotive aerodynamics for 500 deformations of the DrivAer vehicle. Its hybrid Reynolds-averaged Navier–Stokes/large-eddy simulations (RANS–LES) solve incompressible flow with modeled turbulent stresses. The resolved equations can be written as

$$\nabla \cdot \bar{\mathbf{v}} = 0, \quad \rho_f \left(\frac{\partial \bar{\mathbf{v}}}{\partial t} + (\bar{\mathbf{v}} \cdot \nabla) \bar{\mathbf{v}} \right) = -\nabla \bar{p} + \nabla \cdot (2\mu \bar{\mathbf{S}} - \rho_f \boldsymbol{\tau}_t), \quad (\text{B.8})$$

where $\bar{\mathbf{S}}$ is the resolved strain-rate tensor and $\boldsymbol{\tau}_t$ represents unresolved turbulent stresses. We use the surface data at $U_\infty = 38.889 \text{ m s}^{-1}$, with about 8.8 million points per test geometry. Following the available split used with Transolver-3 (Zhou et al., 2026), we use 400 training, 34 validation, and 50 test cases. Separate operators map coordinates and surface normals to pressure and wall shear stress, $\mathcal{G}_{\text{DrivAer}} : (\mathbf{x}, \mathbf{n}) \mapsto (p, \boldsymbol{\tau}_w)$, which are integrated to estimate C_D .

SuperWing. SuperWing (Yang et al., 2025) contains 28,856 flow conditions over 4,239 transonic swept-wing geometries with variations in planform, twist, dihedral, and airfoil shape. The flow is governed by the steady compressible RANS equations,

$$\begin{aligned} \nabla \cdot (\rho_f \mathbf{v}) &= 0, \\ \nabla \cdot (\rho_f \mathbf{v} \otimes \mathbf{v} + p \mathbf{I}) &= \nabla \cdot \boldsymbol{\tau}_{\text{eff}}, \\ \nabla \cdot [(\mathcal{E} + p) \mathbf{v}] &= \nabla \cdot (\boldsymbol{\tau}_{\text{eff}} \mathbf{v} - \mathbf{q}_{\text{eff}}), \end{aligned} \quad (\text{B.9})$$

where the variables denote mean-flow quantities, and $\boldsymbol{\tau}_{\text{eff}}$ and $\mathbf{q}_{\text{eff}}$ include molecular and modeled turbulent transport. ADflow generates the solutions at Mach numbers in $[0.75, 0.90]$ and angles of attack in $[2^\circ, 12^\circ]$, with Reynolds number 2×10^7 and freestream temperature 300 K.

Each wing is represented by $128 \times 256 = 32,768$ surface points. We use 23,366 training, 2,619 validation, and 2,871 test cases, with no wing geometry shared across splits. The operator maps wing geometry, Mach number, and angle of attack to surface pressure and skin-friction coefficients, $\mathcal{G}_{\text{wing}} : (\Gamma, M_\infty, \alpha) \mapsto (C_p, C_{f,\tau}, C_{f,z})$, where τ and z denote the streamwise and spanwise friction directions. These predicted fields determine the drag-to-lift ratio C_D/C_L .

B.3 METRICS

Drag and lift coefficients. Given the aerodynamic force $\mathbf{F}$, drag and lift coefficients are

$$C_D = \frac{\mathbf{F} \cdot \mathbf{e}_D}{q_\infty A_{\text{ref}}}, \quad C_L = \frac{\mathbf{F} \cdot \mathbf{e}_L}{q_\infty A_{\text{ref}}}, \quad q_\infty = \frac{1}{2} \rho_\infty U_\infty^2, \quad (\text{B.10})$$

where $\mathbf{e}_D$ and $\mathbf{e}_L$ are the drag and lift directions, and A_{ref} is the reference area. We reconstruct sampled predictions on the full reference surface before force integration. For density design based on the drag-to-lift ratio C_D/C_L , we exclude cases with near-zero reference lift to avoid unstable ratios. All sampling methods use the same retained cases.

DrivAerML and ShapeNet-Car. Both car benchmarks include pressure and viscous contributions to drag. DrivAerML predicts pressure p and wall shear stress τ_w directly, while ShapeNet-Car estimates the viscous contribution from predicted velocity. Following their respective normal and traction conventions, the implemented coefficients are

$$C_D^{\text{DrivAerML}} = \frac{\sum_i A_i (p_i n_{i,x} - \tau_{w,i,x})}{q_\infty A_{\text{ref}}}, \quad (\text{B.11})$$

$$C_D^{\text{ShapeNet-Car}} = -\frac{\sum_i A_i n_{i,z} (\bar{p}_i + \mu g_{i,z})}{q_\infty A_{\text{ref}}}.$$

Here A_i and $\mathbf{n}_i$ are surface-cell areas and normals. For ShapeNet-Car, $\bar{p}_i$ averages the four vertex pressures, and $g_{i,z}$ is the discrete gradient estimate of the streamwise velocity from the benchmark’s quadrilateral surface stencil. DrivAerML uses its supplied reference area, whereas ShapeNet-Car uses the area of the surface’s convex-hull projection onto the x - y plane. The normalization uses $(\rho_\infty, U_\infty) = (1, 38.889)$ for DrivAerML and $(0.3, 20)$ for ShapeNet-Car, with $\mu = 1.8 \times 10^{-5}$ for the latter.

Airfoil. The inviscid Airfoil benchmark (Li et al., 2023a) uses pressure forces alone. For ordered surface nodes (x_i, y_i) , define $\Delta x_i = x_{i+1} - x_i$, $\Delta y_i = y_{i+1} - y_i$, and $\bar{p}_i = (p_i + p_{i+1})/2$. Trapezoidal integration along the surface gives

$$C_D = -\frac{\sum_{i=1}^{n_s-1} \bar{p}_i \Delta y_i}{q_\infty c}, \quad C_L = \frac{\sum_{i=1}^{n_s-1} \bar{p}_i \Delta x_i}{q_\infty c}, \quad (\text{B.12})$$

where $n_s = 121$, the reference chord is $c = 1$, and $q_\infty = 0.448$. The drag and lift directions coincide with the x and y axes because the angle of attack is zero.

SuperWing. SuperWing (Yang et al., 2025) integrates pressure and skin-friction coefficients on the wing surface. We first undo the stored friction-channel scaling factors of 150 and 300. Using the dataset’s surface-normal convention, the dimensionless force vector is

$$\mathbf{C}_F = \frac{1}{A_{\text{ref}}} \sum_i A_i [C_{p,i} \mathbf{n}_i + (\mathbf{I} - \mathbf{n}_i \mathbf{n}_i^\top) (C_{f,\tau,i} \mathbf{t}_i + C_{f,z,i} \mathbf{e}_z)], \quad (\text{B.13})$$

where $\mathbf{t}_i$ is the local chordwise direction in the x - y plane, and A_{ref} is the half-wing reference area. The tangential projection converts the stored friction components into surface traction. Rotating by the angle of attack α gives

$$C_D = C_{F,x} \cos \alpha + C_{F,y} \sin \alpha, \quad C_L = -C_{F,x} \sin \alpha + C_{F,y} \cos \alpha. \quad (\text{B.14})$$

B.4 TRAINING DETAILS

All backbones follow the Transolver architecture (Wu et al., 2024), with eight Physics-Attention blocks and eight attention heads per block. Table 3 summarizes the task-specific model and training parameters. We use AdamW with weight decay 10^{-5} and gradient-norm clipping at 0.1. Experiments run on a server equipped with eight NVIDIA H20 GPUs and two Intel Xeon Platinum 8457C processors, providing 96 physical CPU cores and 192 hardware threads.

Table 3: Backbone configurations for the eight benchmarks. Batch sizes are global numbers of cases per optimization step. Learning rates are the initial rate for cosine decay and the peak rate for OneCycle. Sampling bounds are inclusive point counts.

Parameter	Standard PDE benchmarks					Industrial applications		
	Elasticity	Plasticity	Airfoil	Pipe	Darcy	ShapeNet-Car	DrivAerML	SuperWing
Hidden width	128	128	128	128	128	256	256	128
Physics states	64	64	64	64	64	32	32	64
MLP ratio	2	1	2	2	2	2	2	2
Learning rate	10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-3}	2×10^{-3}	10^{-3}
Epochs	500	1,000	1,000	1,000	1,000	400	1,000	1,000
Batch size	1	8	8	4	4	2	4	16
Scheduler	Cosine	OneCycle	OneCycle	OneCycle	OneCycle	OneCycle	OneCycle	OneCycle
Loss	Rel. L_2	Rel. L_2	Rel. L_2	Rel. L_2	Rel. L_2	Dual MSE	Rel. L_2	Rel. L_2
Min. points	389	1,252	4,508	6,656	7,225	12,874	85,900	9,830
Max. points	972	3,131	11,271	16,641	7,225	32,186	859,000	32,768

We adopt geometry-amortized training (Zhou et al., 2026) by resampling a uniform subset of mesh points at each optimization step. The subset size is drawn uniformly from the integer interval specified in Table 3, and points are sampled without replacement. The lower bound is $\text{round}(0.4N)$ for Elasticity, Plasticity, Airfoil, Pipe, and ShapeNet-Car, and $\text{round}(0.3N)$ for SuperWing, where N is the full-mesh point count. DrivAerML uses the absolute range $[85,900, 859,000]$ for both its pressure and wall-shear-stress models. Darcy is trained on the full mesh. Each backbone remains frozen during sampling-policy evaluation.

DrivAerML uses four workers with one case per worker, giving a global batch size of four and a peak learning rate of $10^{-3}\sqrt{4} = 2 \times 10^{-3}$. Training minimizes relative L_2 error except on ShapeNet-Car, which uses velocity MSE plus 0.5 times surface-pressure MSE in normalized output space.